Real-world (compare your loop to the library's) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `agent_executor_comparison_practice.py`: now that you've built the loop, `create_agent` should look familiar — the same think → act → observe turns, pre-built and tested. Running both on the same prompts shows you exactly what the library does for you. **If not:** you'd either trust the library blindly, or rebuild it every time without knowing it already exists.

Needs `pip install langchain langgraph`, the Doc06 copies in `practice/`, and the Intermediate exercise's `react_loop_practice.py`.

Basic Version

Approach 1 — the library, once

```
# agent_executor_comparison_practice.py
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

from tools import ALL_TOOLS

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
agent = create_agent(model, tools=ALL_TOOLS)

inputs = {"messages": [{"role": "user", "content": "What's 5 + 7?"}]}
result = agent.invoke(inputs)
# how: the result holds every message of the run; the last one is the answer
print("Final answer:", result["messages"][-1].content)
```

Expected output (wording varies):

```
Final answer: 5 + 7 is 12.
```

It works, but you can't yet say whether it did the same thing your loop would have done.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — both engines, the same prompts, and a run that never ends

```
# agent_executor_comparison_practice.py
from langchain.agents import create_agent
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langgraph.errors import GraphRecursionError

from react_loop_practice import MaxIterationsExceeded, run_agent
from tools import ALL_TOOLS

PROMPTS = [
    "What's 5 + 7?",
    "What's the capital of France?",
    "Add 1 + 2, then add 3 to that.",
]


def count_library_tool_calls(messages: list) -> int:
    # how: every tool result in the run is a message with type "tool"
    count = 0
    for message in messages:
        if message.type == "tool":
            count = count + 1
    return count


@tool
def always_ask_again(city: str) -> str:
    """Look up the weather for a city. Always call it again to check."""
    return "Not sure yet - call this tool again."


def main() -> None:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    agent = create_agent(model, tools=ALL_TOOLS)

    for prompt in PROMPTS:
        mine = run_agent(prompt)
```

```
        inputs = {"messages": [{"role": "user", "content": prompt}]}
        library = agent.invoke(inputs)
        library_calls = count_library_tool_calls(library["messages"])
        print(prompt)
        print(f"  mine:    {len(mine.steps)} tool calls")
        print(f"  library: {library_calls} tool calls")

    # why: both loops must stop a run that never ends — compare how
    looping_agent = create_agent(model, tools=[always_ask_again])
    inputs = {"messages": [{"role": "user", "content": "Weather in Paris?"}]}
    try:
        # how: recursion_limit counts graph steps — about 2 per turn
        looping_agent.invoke(inputs, {"recursion_limit": 6})
    except GraphRecursionError:
        print("library: GraphRecursionError at recursion_limit=6")
    try:
        run_agent("Add 1 + 2, then add 3 to that.", max_iterations=1)
    except MaxIterationsExceeded as e:
        print(f"mine:    MaxIterationsExceeded, {len(e.steps)} step logged")


if __name__ == "__main__":
    main()
```

Expected output (counts can vary a little between runs):

```
What's 5 + 7?
  mine:    1 tool calls
  library: 1 tool calls
What's the capital of France?
  mine:    0 tool calls
  library: 0 tool calls
Add 1 + 2, then add 3 to that.
  mine:    2 tool calls
  library: 2 tool calls
library: GraphRecursionError at recursion_limit=6
mine:    MaxIterationsExceeded, 1 step logged
```

Difference from Basic: the same three prompts through both engines, compared by tool-call count, and the same "never ends" case through both. The library raises `GraphRecursionError` without a step log; yours raises `MaxIterationsExceeded` carrying it.

Back to the exercise · Hint 1 · Hint 2 · Solution

Approach 2 — watch the library's steps as they happen

Story — `agent.stream(..., stream_mode="updates")`: counting tool messages after the run tells you *how many*; streaming the updates shows you *which step did what*, in order — the library's version of your `steps` list. **If not:** the library's loop would stay a black box that only shows its final answer.

```
# agent_executor_comparison_practice.py
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

from tools import ALL_TOOLS

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
agent = create_agent(model, tools=ALL_TOOLS)
inputs = {"messages": [{"role": "user", "content": "What's 5 + 7?"}]}

# how: stream_mode="updates" hands over each graph step as it happens —
# one dict per step, keyed by the step's name ("model" or "tools")
for update in agent.stream(inputs, stream_mode="updates"):
    for step_name in update:
        last = update[step_name]["messages"][-1]
        print(f"{step_name}: {last.type} -> {last.content}")
```

Expected output (wording varies):

```
model: ai ->
tools: tool -> 12
model: ai -> 5 + 7 is 12.
```

The first `model` step has empty content because it only asked for a tool; the `tools` step is the Observation; the last `model` step is the answer — your think → act → observe, printed by the library.

Which one should you actually use? Both have a place: `create_agent` when the default loop is enough; your own loop when you need your own log, your own error, or your own stopping rule — which is why Project 2's Worker is hand-built.